

R packages development training

Mathieu Depetris

Table of contents

Welcome!	4
Introduction	5
I Training framework and requirements	6
1 - Operating system and working environment associated	7
1.1 Requirements software	7
1.2 Additional software	9
2 - Overview of R packages used	12
3 - Before go down the rabbit hole	17
II First part - Global package architecture	18
4 - Before create anything	19
4.1 First reflexion, the perimeter	19
4.2 Second reflexion, the name	19
4.3 Third reflexion, the logo	21
5 - Creation of the package structure	22
5.1 Initiation the global architecture	22
5.2 Overview of the package strcutre	24
5.2.1 The package directory	24
5.3 RStudio package environment	25
6 - Package states	27
6.1 Source package	27
6.2 Bundled package	27
6.3 Binary package	28
6.4 Installed package	28
6.5 In-memory package	29

III Second part - Package metadata	30
7 - The DESCRIPTION file	31
7.1 Fields Package, Title and Description	32
7.2 Field Version	32
7.3 Field Author	32
7.4 Field License	35
7.5 Field Encoding	35
7.6 Roxygen fields	35
7.7 Futures fields	36
8 - Licensing	37
8.1 Global overview	37
8.2 Software development licenses	37
IV Third part - Adding code	39
V Fourth part - Tests and checking	40
VI Fifth part - Documentation	41
VII Sixth part - Current management	42
References	43
Appendices	44
A Not define yet	44

Welcome!

Welcome to the online support to learn how to organise your R code through the creation of an R package. You will find that this process is easier than you can imagine and with a little bit of pratiques you will be able to create your own package. In addition, you will discover that this way open you the field to develop reproducible code, documented faster and simply your processes at many levels and learn how other people can download and use it.

All this website and material associated if free to use, licenced under the [CC BY-NC-ND 4.0](#) licence. At least you will find below several training sessions available if you want a guided explanation for R package creation.

To conclude, this support was developed according to the book *R Packages (2e)* [1] written by [Hadley Wickham](#) and [Jennifer Bryan](#). An online version is available through this [link](#), don't hesitate to take a look.

Introduction

To do

Part I

Training framework and requirements

1 - Operating system and working environment associated

Before going into the fun of R package development, take a moment to setup our working environment and overview some interesting software which can simplify our life.

First, all the training and test that we will do among this book will be executed with Windows Operating System (OS). When you develop your package, one of the aims could be to share or propose your work to other peoples and you have to take into account about the variability of the processes among OS, software for example R dependencies. We will put in place several processes to support us in our task but keep in your mind that processes that work on an OS could not work in the same pattern in another one. Futermore, one of our tasks among the lifecycle of our package will be to maintain a maximum of compatibility, for example through integration of update or evolution in our code.

Developing R packages assume that you have a little experience with the R software and not start from scratch with it. Even if you see that R package development is not so hard that it looks like, you will find a lot of resources on the internet to support you if you want to start with R and develop your skills in it.

1.1 Requirements software



For the training we will use the *R* version 4.5.0 for Windows available through the web page of [The Comprehensive R Archive Network](#). You will find also precompiled binary distributions for Linux and macOS.



In addition, we will use the Integrated Development Environment (IDE) *RStudio*. Today this software is one of the most used in the R communities and includes a lot of things like a console, syntax-highlighting editor that supports direct code execution, and tools for plotting, history, debugging, and workspace management. Furthermore, recent evolution like the creation of [Posit](#) compagny make the software ecosystem more friendly connected with R and Python. For the training we will use the RStudio version 2025.05.0+496 “Mariposa Orchid” for Windows OS. You can download the one for your configuration throught the [Posit download webpage](#).



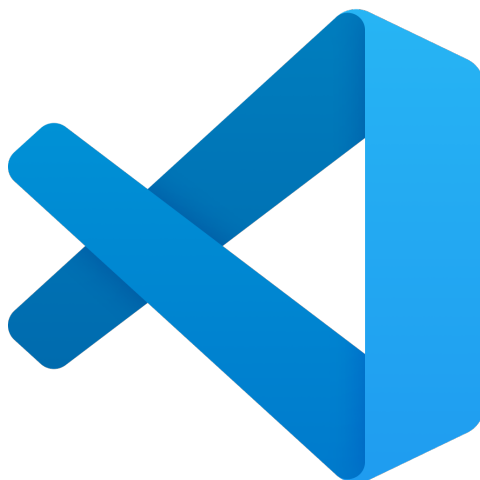
At least, we will use Git, for version control system, to host our R package and apply several processes of Continuous Integration (CI), most of them already prepared by the R communities. Today, this is a very precious tool that helps developers track changes in their code, collaborate efficiently, and manage different versions of a project. During this training we will use one of them. You can download the software on the official website through this [link](#).

! Important

Under Windows OS, it's necessary to install the [RTools](#) software. This is a toolchain for building R and R packages from source on Windows. And don't need it on macOS and Linux because these OS have usually pre-installed compilation tools. You can download it from the [official link](#). Be aware that you need to take the correct version according to our R version installed (for this training it would be the RTools 4.5).

1.2 Additional software

Several software are not necessary to create your own R package but bring you a lot of improvement and at least simplify your life.



The first one is [Visual Studio Code](#) (or VS Code) which is a free, lightweight, extensible code editor developed by Microsoft. He supports a wide variety of programming languages such as R, JavaScript, Python, C++, Java, HTML/CSS. In addition is cross-platform compatibility, support Git and other version control systems and one of his best advantages is to provide extensions to add functionality and customize the environment. If you want you can download it from this [official website](#) and if it's possible according to your configuration prefer a system installer version. For this training I will use the version 1.100.2 on Windows OS.



According to the use of Git, we will use one of them call [GitHub](#). You will see that a lot of processes were developed for work natively with GitHub and facilitate your R package creation. If you want to use another one, like a GitLab hosted by your company for example, you can of

course but you will have to modify and adapted by yourself several processes propose during this training. In addition we will use here a Graphical User Interface (GUI) for managing Git repositories called GitHub Desktop. This is a free, open-source application which simplifies current Git operations, making it easier for developers to clone repositories, commit changes, create branches, and push updates without using the command line. Like its name suggests is working natively with GitHub repositories but you can also use it for connection with other Git like GitLab. Officially is available for Windows and MacOS but you can also find adapted version for Linux on the internet. For download it goes through the following [link](#).

2 - Overview of R packages used

This section aim to introduce brively which R packages environment we will use. Since several years, the R communities evolve in many interesting ways and today if you want to write your own package there are a lot of support that you can find.

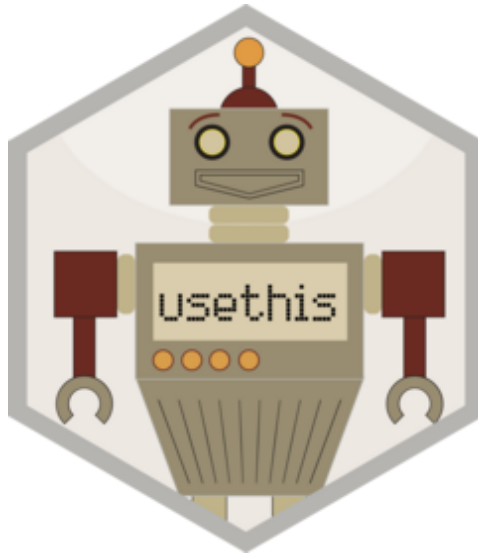


First of all, it's difficult to talk about package without talk about the [tidyverse](#) approach. Tidyverse is a [collection of R packages](#) designed for data science. They share an underlying design philosophy, grammar, and data structures. If you use R, it's almost sure that you have already use this kind of packages, maybe even without knowing them. Even if there are none mandatory for R use, these packages bring a lot of benefices and at the end make data science easier, faster, and more fun (yes, you will see).

So far we don't go deeper into all the useful R packages of the tidyverse collection, but only focusing on these related to the R packages development.



The first one is called [devtools](#) [2]. The aim of this package is to make R package development easier by providing R functions that simplify and expedite common tasks. In practical, if you use the Rstudio IDE you will not interact directly with this package because the majority of common processes will be integrated automatically in the Rstudio environment and become “click button” processes.



The second one is the sub-package you are most likely to interact with directly. It be called [uethis](#) [3] and is aim is to automates repetitive tasks that arise during project setup and development. The idea is to, if it's possible, use functions available by the package, rather that modify directely the code, to reduce error and be sure to have the last version of process strucutre.





There are two more important packages related to design of the documentation: [roxygen2](#) [4] and [pkgdown](#) [5]. With the first one through the use of tags, the code and the documentation are adjacent and dynamically inspects the objects that it's documenting. We will go back to them into the following sections but keep in your mind that theses two tools simplify and automate the documentation creation and evolution.



The last one is called `testthat` [6] and focusing on the field of testing your code, which can be painful and tedious, but it greatly increases the quality of your code.

In addition we will use or talk about several other packages, for example `lintr` [7] or `renv` [8], but it's more a case-by-case utilisation and we will discuss them when the time comes.

3 - Before go down the rabbit hole

Before we start to developing our first R package, let's talk briefly about the aim to organise your code into a package and in fact why we consume time to develop this kind of structure.

When you use R, a package is the fundamental unit of shareable codes and brings a lot of advantage, like regroup in one place all the stuff (code, data, documentation for example), improve the standardisation of your process and the sharing them easier. In fact, build an R package is nothing more than a way of organising things that you have already done by another way and facilitate at the end the life cycle of your processes.

Furthermore, you will see quickly that you can use a lot of processes already developed ,and update in time, by the R communities. There are package that make package, processes that allow to automated tasks and the tidyverse philosophy explain in the previous section help a lot.

To summarise:

- don't worry, you will see that all this process is painless and you will be able to create very great thing in brief time,
- in addition, you will save time! you will follow a template and automate a lot of processes with the rule "anything that can be automated, should be automated",
- to conclude, you will use standardised conventions that open to you the path a several free and evolving standardised tools.

Part II

First part - Global package architecture

4 - Before create anything

Before starting playing with functions and create the global structure of your package, the best thing to do is to think a little bit about what you want to create. The idea is not to spend a very long time to do that, but if you take this time (especially is it's the first time) you will see that at the end you will gain. The majority of the package component can be modified after definition, but some update could be heavy and painful and sometimes it's simple to avoid that by taking a moment of reflection.

4.1 First reflexion, the perimeter

The aim here is to answer the question, what I want to put into my package or what I want my package will do? You don't have to be exhaustive (if you can it's better) but try to explore several fields like:

- If I have several functions already developed (even partially), it's logical to put all of them into a single structure?
- If my package will be used by other users, does the process of use if relevant and logical for users?
- If I think bigger, do you think my package could be placed in specific environment and maybe interact with other packages? A good example of that is if a package interact with other packages and each one has a specific purpose (for example one dedicates to the controls and another dedicates to the graphic display).

You can image other cases and again it's normal if your perimeter evolve with time regarding the maturity of your work. A good solution could be to design flux diagrams for example explain (even for you) the process and maybe associated one. In addition, these kinds of figures could be useful for the documentation process. A quick research on the internet will be displaying several software, free and paid, for doing that.

4.2 Second reflexion, the name

Now you know globally the purpose of my package. The next step is to define is named. It's an important moment because the name is one of the first things that you and the others will see. At a time where the referencing (like the SEO or Search Engine Optimisation) and

visibility is important items regarding the life of your work, it could be bad to lose points at this moment just because the name is not very clear and friendly.

Globally, I advise you to follow several good praticals:

- the name has to be **simple** and **clearness**. What's better than a name that's easy to understand and remember and ideally allows a reading to understand what it does.
- avoid **special characters**, like the accents or dot, and prefer use convention like Snake Case (each word is separated by and underscore character) or at least Camel Case (start with the first word with lowercase and capitalize the first letter of each word that follows without space).
- insure the **compatibility** of the name with package repositories like the CRAN ([Comprehensive R Archive Network](#)) or [Bioconductor](#).

To support you in this step, you can use functions of the R package, [pak](#) [9].

This package is a modern alternative to the R standard function, `install.packages()`. Its goal is to make package installation faster and more reliable. In particular, it performs all HTTP operations in parallel, so metadata resolution and package downloads are fast. In addition, it has a dependency solver, so it finds version conflicts before performing the installation. Furthermore, it supports CRAN, 'Bioconductor' and 'GitHub' packages as well.

```
package_name <- "lightning"
library(pak)
pkg_name_check(name = package_name)
```

-- lightning --

valid name	CRAN	Bioconductor	not a profanity
------------	------	--------------	-----------------

Wikipedia

Lightning Lightning is a natural phenomenon consisting of electrostatic discharges occurring through the atmosphere between two electrically charged regions. One or both regions are within the atmosphere, with the second region sometimes occurring on the ground. Following the lightning, the regions become partially or wholly electrically neutralized.

...

<https://en.wikipedia.org/wiki/Lightning>

Wiktionary

lightning Wikiversity

Etymology: From light(e)n + -ing. Doublet of lightening.

Noun: lightning (usually uncountable, plural lightnings)
A flash of light produced by short-duration, high-voltage discharge of electricity within a cloud, between clouds, or between a cloud and the earth.

...

<https://en.wiktionary.org/wiki/lightning>

Sentiment: :| (0)

4.3 Third reflexion, the logo

Even if it's not mandatory, what is more fun than you have your own package logo. If you are familiar with the tidyverse R package communities, you should see that each package has its own logo with some similar structure of hex stickers.

If you want to design your own logo:

- the convention use for tidyverse package is to orient the logo with a vertex at the top and bottom, with flat vertical sides.
- take of look on the [hexbin](#) community website when you have a lot of hexagon stickers and some recommendation for the “[best sticker size](#)” (figure 1).

Figure 1: Standard dimensions of a hex sticker

- The [hexSticker](#) package [10] helps you make your logo from within the comfort of R.
- Another good solution is to use an AI image generator. Without open the field of which one you have to select, according to several considerations like the integrity of the processes and the ecological impact of that, this kind of software be able to generate your logo easily according to your preference.

If you have one at the moment, keep it closely and we will see how to integrate it in our package structure. If not don't worry, you can go back to this process further.

5 - Creation of the package structure

5.1 Initiation the global architecture

Now you have an working environment correctly configured and we have taken a few moment to thing about our package aim. We can go deeper and initiate the structure of our new package from an active R session. You don't have to take care about the configuration of your local session because the function associated will be create a clean new project for your package. First, load the [devtools](#) and by association is friend [usethis](#).

```
library(devtools)
```

Le chargement a nécessité le package : usethis

Just to be sure that you have everything you need to develop R package you can launch the following command line to display a report of the package development situation.

```
dev_sitrep()
```

```
-- R -----
* version: 4.5.0
* path: 'C:/Program Files/R/R-4.5.0/'

* R is out of date (4.5.0 vs 4.5.1)

-- Rtools -----
* path:
-- devtools -----
* version: 2.4.5

* devtools or its dependencies out of date:
  'xml2', 'curl', 'rprojroot', 'xfun', 'later'
Update them with `devtools::update_packages("devtools")`
```

```
-- dev package -----
* package: <unset>
* path: <unset>
```

If everything rock's, launch the next command to initiate the global architecture of your package. For the training, my new R package will have the acronym "macfly" for "My Anwesone paCkage For Learning with Yearning".

```
create_package(path = "~/macfly")
```

```
v Setting active project to "C:/Users/mdepetri/Documents/macfly".
```

```
i Leaving 'DESCRIPTION' unchanged.
```

```
Package: macfly
Title: What the Package Does (One Line, Title Case)
Version: 0.0.0.9000
Authors@R (parsed):
  * First Last <first.last@example.com> [aut, cre]
Description: What the package does (one paragraph).
License: `use_mit_license()`, `use_gpl3_license()` or friends to pick a
  license
Encoding: UTF-8
Roxygen: list(markdown = TRUE)
RoxygenNote: 7.3.2
```

```
v Setting active project to "<no active project>".
```

Another solution with RStudio is to use the interface through the *file* menu, *New Project...*, *New Directory*, *R Package*, file the field "Package name:", click on "Open in new session" and finish with *Create Project". Even is the approach is similar, the click-button interface create in addition a template function with a proposal for the documentation associated. In opposite, the command line creation include by default a .gitignore file (for Git use). It's not true important but it could be a bit better to use the command line because we will learn how to create a R function template later and in your developer life it could be very rare (and dangerous) when you don't use a Git associated to your work.

By default, RStudio open a fresh session associated to the R environment of your package and you should need to call again `library(devtools)`.

i Note

For your information, you will find in the [usethis](#) package a function `create_tidy_package()` that extends `create_package()` by applying many of the tidyverse development conventions as possible. I don't recommend using that way, instead of you know exactly what you do and to save time in the future, because it's better to really understand each step of the package creation.

! Important

At least, you could see sometime a function call `package.skeleton` from the [utils](#) R package. Even if this function create a package structure it's an "old way" to do and you should have some trouble if you use it, especially with you want to use the tidyverse philosophy.

5.2 Overview of the package structure

If you follow the previous step, you should have the opening of a new RStudio session and the creation of a new directory on your local drive called just like your package name. Let take a moment that.

5.2.1 The package directory

Inside the package directory, you should see 8 elements (the `.Rhistory` file is normally generated when you close for the first time in your RStudio session, but his generation depends on your personal configuration).

i Note

If you show a number or kinds of elements different than below without change anything in the command line below, maybe it's because you don't have to unlock the hidden files. In order to remedy you can go in the setting directory option (normaly and can access to that directly in any directory on Windows) and in the parameters apply the visibility of hidden elements. In RStudio to see hidden files in the *files* panel, go to *More* (gear symbol) the tick *Show Hidden Files*.

In addition, if highly recommend displaying file extension in your file explorer (on Windows, it's normally near the option for hidden elements). By default this display is hidden and could be a little bit strange when you activated it, but it's very important to sensitize the user of what kind of file you will open. Furthermore, be aware the "clothes don't

make the man” and the extension is only a shortcut for the system to know how to deal with that but don’t make any guarantee of what you have inside. In accordance with rules of good practice (especially link to the security), take a moment to think about what you’re about to open and display the extension is a good first step to begging.

Element name	Type
.gitignore	file
.Rbuildignore	file
.Rproj.user	directory
DESCRIPTION	file
macfly.Rproj	file
NAMESPACE	file
R	directory

- *.gitignore*, we will see in the next step (put here section id) how to connect our local package directory to a git. This file indicates which files or directories have to be ignored for the git commits.
- *.Rbuildignore*, in the same way that the *.gitignore*, this file indicate which elements have to be excluded during the package compilation.
- *.Rhistory*, by default this file keeps tracks of every command you’ve typed into the R console. When you close R, it often saves this file automatically. Be aware of that process, because this file may contain sensitive data (access to local files, credentials, etc.). You will see later that when we make connection with Git, we will ignore this file through the *.gitignore* file. If you don’t want to generate and keep this file, in RStudio *Global Options*, section *General* you can disable “Always save history (even when not saving .RData)”.
- *.Rproj.user*, this directory is normaly hidden and used internally by RStudio.
- *DESCRIPTION*, this is an important file that provides metadata about your package, like the authors, the package description and the dependencies.
- *macfly.Rproj*, this file, dedicate to RStudio, aims to organize your work through define the working directory of your project, memorize parameters specifics and able to open directly the project in RStudio by double click on it.
- *NAMESPACE*, this file aims to declares functions “used” among your package (through external and internal ways).
- *R*, this directory will contains all the functions developed in your package.

5.3 RStudio package environment

If you take a look at the panel *Build* in the Rstudio environment you should see several new sections, like *Install* or *Check* (figure 1). We will take a look after in each of one, but keep in

our mind that this sections is a shortcut way to apply processes on your package.

Figure 1: RStudio Build panel

i Note

In preparation for the documentation generation, the only thing to do now is to go to the *Configure Build Tools* options (click on *More* (gear symbol) and *Configure Build Tools*). In the *Project Options* menu, *Build Tools* section, click on the *Configure* button near *Generate document with Roxygen*. In the new windows, fill all the proposals. This process will create and update documentation automatically when you launch several packages processes. Sometimes, it may justify not filled all is if you have a really huge documentation or a specific rule for documentation generations.

6 - Package states

Now we have created our package structure, it's important to take a moment to understand exactly the different states that your package can have and in addition make a link with all the things that you already do, like use the `install.packages()` function, maybe without knowing what is behind that. There are five different package states that we will explain below.

6.1 Source package

When we create our package in section 5, we create a source structure package. This is not the format you are most familiar with but to summarize it's just a directory with files and subdirectories, grouped in a specific structure. If you have already interact with packages developed through a Git (the majority of packages have a Git repository dedicated), the structure associated with it is a source package.

General, the source package is the most updated form of a package but be aware because it's mean not necessary the most stable one. To conclude, keep in our mind that this state is not associated to any OS, it's just an element with a particular organisation.

6.2 Bundled package

A bundled package is just a package compressed into a single file. By convention, R package extension is `.tar.gz`. The first one (`.tar`) is for the reduction in one single file, and the second one (`.gz`) is compressed with [gzip](#). Like the previous state, it's again not associated to any OS and it's more a means of transportation for your package. If you are familiar with the package available on CRAN, you should see in a package description page on the CRAN website the item *Package source* with an associated `.tar.gz` file, which is the bundle version of the package.

If you decompress a bundle, you will find 3 differences with a source package structure:

- If you have vignettes (we will discuss that during the section of the documentation), they have been rendered and appear below the repositories `inst/doc/` with an index in the build repository.

- During the development phase, you can use temporary files for example increase the compilation time. These files are never found in a bundle.
- To finish, all the files listed into the `.Rbuildignore` of a source package is not included in the bundle because by definition there are excluded from the package compilation.

6.3 Binary package

This next state is the first one where you have the integration of the OS specification. If you want to share your package without any package development tools mandatory for the user, you need to provide it with a binary package. Like a bundled package, a binary package is a single file but as a user you have to choose the correct one for your OS. Normally you should find two kinds of binary package extensions, one `.tgz` for macOS and another `.zip` for Windows. On Linux, you have to install tools necessary to `.tar.gz` files but some recent resource like [Posit Public Package Manager](#) provide to Linux user access to binary packages just like the other OS.

We don't go deeper into the internal structure of a binary package but be aware that its structure is very different than a source and bundled packages and if you want to go deeper you can find a very nice comparative between the three [here](#).

6.4 Installed package

An installed package is a binary package that's been decompressed into a package library (see section below). This is what happens when you use the function `install.packages()`. At this moment and it's practically sure that you have already some troubles in the past, it could be complicated to reinstall a package already used in the current R session. Sometime you have popups associated to "need of compilation", restart a fresh session or again question related to take a package from sources because the version is more updated than the one that you have selected. There don't have a perfect solution by when troubleshooting, Windows users should strive to install packages in a clean R session, with as few packages loaded as possible. Regarding the popup question of taking the package from the source because the version is higher, I suggest doing that only if you know what you are doing, for example if you want a specific process or function developed so far only on the source.

If you have a lot of troubles with packages installations and update, remind of the [pak](#) [9] package. We used it before to check our package name, but he provides a good and powerful alternative to `install.packages()`. Use it with caution because it is a relative newcomer, but we are certainly using it more and more in our personal workflows (in the GitHub action that we see after for example).

6.5 In-memory package

Everyone how used R used this state all the time, thought the command `library()`. When you use that, you loaded the package associated into the memory (and for the R session) and in addition, attached the package to the search path. We will see after the difference between loading and attaching package when we talk about the “hell” of dependencies (it’s a very important thing when you writing package).

To conclude this part, did not confuse the word *library* and *package*. Sometime people talk about a package as a library with for example “I use the library `DBI` to dealing with my database”. A library is just like if the name suggests an element which containing installed packages, just like a library “in the world” contain books. Most of the time it is not a problem because people will understand anyway, but the distinction is important and useful as you develop package.

Part III

Second part - Package metadata

7 - The DESCRIPTION file

The file created initially when we launch the creation of our package structure contains the major metadata information. You will find fields like the name of the package, the title and description associated, version or authors. The format of this file is DCF (Debian Control Format). It looks like the YAML on several points, each line consists of a field name and a value, separated by a colon. When you have a value display on multiple lines, like the field `Description`, the line following the first one needs to be indented (you will see that in the example below). It's a plain text file and like another one you can open it through a text editor (like NotePad or Visual Studio Code).

For our package created before, this is the initial DESCRIPTION file.

```
Package: macfly
Title: My Anwesone paCkage For Learning with Yearning
Version: 0.0.0.9000
Authors@R: c(
  person("Mathieu", "Depetris", , "mathieu.depetris@cnrs.fr", role = c("cre", "aut"),
    comment = c(ORCID = "0000-0001-8080-0531")),
  person("UMR MARBEC", "MARine Biodiversity Exploitation & Conservation", role = "cph")
)
Description: An example of R package development with functions and all
  the stuff around to increase the joy of writing our own processes.
License: `use_mit_license()`, `use_gpl3_license()` or friends to pick a
  license
Encoding: UTF-8
Roxygen: list(markdown = TRUE)
RoxygenNote: 7.3.2
```

To update this file you have two possibilities, open the file and modify it by hand or use the R package `desc` [11] to manipulate it by command lines. In the following part, we will use functions of this package to reduce as possible typing error.

7.1 Fields Package, Title and Description

The first line in the description file is the **Package** field and refer to the name of the package that we define before (Section 4.2). You don't have anything to modify in this field.

Like name suggest, **Title** is one line description of the package. It should be plain text (no markup), capitalized like a title, and not end in a period. Shorter is better and anyway, listings will often truncate the title to 65 characters.

Description field is one (and only one) paragraph more detailed than the **Title** field. You can use multiple sentences and it's recommended to use multiple lines. Each line must be no more than 80 characters wide and indent the new line with 4 spaces.

It's important to find a good title and description because it's something that people see in first, especially if you submit your package on CRAN (it's display on the package CRAN website).

For our example, I used the code below. You can modify it according to our needs.

```
library(desc)
# By default the function read the DESCRIPTION file of the current working directory
description_file <- desc()
# For modify the title
description_file$set(Title = "My Anwesone paCkage For Learning with Yearning",
                     Description = "An example of R package development with functions and a")
```

7.2 Field Version

This field is a very important way to indicate the state of your package and evolving across the time and more precisely is lifecycle. We will go deeper into this in the [section to define].

7.3 Field Author

The field **Authors@R** inside the DESCRIPTION file identify package's authors and contributors. This is a very important information because with that you know whom to contact if you are any troubles with the package. In comparison with the other field, this one contains executable R code rather than plain text. You could find also a declaration of authors like below.

```
Author: FirstName LastName [aut, cre]
```

It's the classic way but I highly recommend using the modern one with **Authors@R** because you will be able to increase the amount of information inside, like the affiliations or the ORCID number. Furthermore, it's the format recommended by CRAN. In addition, it integrates the function `person()` which holds information about individuals such as their name and email address and displays them in the proper way in R.

You can use several `person()` function's arguments (take a look in the documentation to have a full overview) but you will find below the most useful and common:

- **given** and **family** specify respectively the first name and the last name. For a non-person entity, like your research unit, use only **given** and omit the **family** (except for special purpose, like if you want to have the acronym of your unit and the full name),
- **email** is very important because it's the logical way to contact for the user if he has any question on your package and processes associated. This field is in relation to the **role** below and condition if it is a mandatory information or not,
- the **role** field indicates the contribution and the implication of each author in the package development and life. In practice you can have multiple roles for one person and you can choose between these:
 - the **cre** is the creator or the maintainer of the package. You can have only one and it's the person of the entity that you have to contact if you have troubles. In relation with comment before, this field has to be associated with a valid email. Furthermore, despite the name, this role is used for the current maintainer of the package even if it's not the initial creator of the package,
 - **aut** is for authors and indicates peoples which have significant contributions to the package,
 - **ctb** is for contributors like people which made smaller contributions like patches or reviewers,
 - **cph** is copyright holder. Typically your research unit should be a copyright holder.
 - Sometime you can have the **fnd** role for funder, which is like the name suggests, people or organizations that have provided financial support to the package.
- **comment** is the last one useful because it lets you the opportunity to include your [ORCID identifiers](#).

At least, your package should have one author and one maintainer (it could be the same person), with the last one associated to a valid email. In addition, you can list multiple arguments (like several roles for one person) or multiple authors with a `c()`.

According to the section before, you will find below an example for updating the field **Author** of the package training example:

```
# Removing default "Author" field
description_file$del("Authors@R")
# Adding authors
```

```
description_file$add_author(given = "Mathieu",
                             family = "Depetris",
                             email = "mathieu.depetris@cnrs.fr",
                             role = c("cre", "aut"),
                             orcid = "0000-0001-8080-0531")
description_file$add_author(given = "UMR MARBEC",
                             family = "MARine Biodiversity Exploitation & Conservation",
                             role = c("cph"))
```

At least, theses fields are used to generate basic citation for our package and you can use the function `citation()` for display it. You can find below the exemple of citation for the package [dplyr](#).

Pour citer le package 'dplyr' dans une publication, utilisez :

```
Wickham H, François R, Henry L, Müller K, Vaughan D (2023). _dplyr: A
Grammar of Data Manipulation_. doi:10.32614/CRAN.package.dplyr
<https://doi.org/10.32614/CRAN.package.dplyr>, R package version
1.1.4, <https://CRAN.R-project.org/package=dplyr>.
```

Une entrée BibTeX pour les utilisateurs LaTeX est

```
@Manual{,
  title = {dplyr: A Grammar of Data Manipulation},
  author = {Hadley Wickham and Romain François and Lionel Henry and Kirill Müller and David
  year = {2023},
  note = {R package version 1.1.4},
  url = {https://CRAN.R-project.org/package=dplyr},
  doi = {10.32614/CRAN.package.dplyr},
}
```

So far, we don't modify any other lines in the DESCRIPTION file. The next modifications will be appear directly in the file when you will launch the r functions associated. So the last thing to do is to save the DESCRIPTION file modifications by erasing the previous one. Use the following command line do doing that.

```
# Update, by remplacement, the DESCRIPTION file (in the current working directory)
description_file$write()
# Bonus, this function standardize the DESCRIPTION file (ordering fields and dependencies)
usethis::use_tidy_description()
```

```

Package: macfly
Title: My Anwesone paCkage For Learning with Yearning
Version: 0.0.0.9000
Authors@R: c(
  person("Mathieu", "Depetris", , "mathieu.depetris@cnrs.fr", role = c("cre", "aut"),
    comment = c(ORCID = "0000-0001-8080-0531")),
  person("UMR MARBEC", "MARine Biodiversity Exploitation & Conservation", role = "cph")
)
Description: An example of R package development with functions and all
  the stuff around to increase the joy of writing our own processes.
License: `use_mit_license()`, `use_gpl3_license()` or friends to pick a
  license
Encoding: UTF-8
Roxygen: list(markdown = TRUE)
RoxygenNote: 7.3.2

```

7.4 Field License

This field is mandatory and reflect your package's license. To simplify the thing, the license will inform users of your package what they can do, or not, when the use or manipulate your package. This is an important task and you have to ask yourself about several question to take the better decsion according to your needs. You can find a deeper reflexion on this aspect in the [section to define].

7.5 Field Encoding

If you have special characters or diacritical marks like accents in your data, you have already dealing before with the encoding, to avoid text warp. By default the field **Encoding** is UTF-8 as this is the most common encoding in use today. Normally you don't have to modify.

7.6 Roxygen fields

By using the function `create_package()` before, two fields related to documentation will be added in the description file, **Roxygen** and **RoxygenNote**. We will see later (section to define) what they are connected to but there are in relation to the [roxygen2](#) package, your best ally to simplify the documenting production and icing on the cake theses fields will be updated automatically in the DESCRITPION according to actions initiate during the package development and lifecycle.

7.7 Futures fields

You will that several new fields will be added in the DESCRIPTION file, like `URL`, `BugReports`, `Imports` or `VignetteBuilder`. We will discuss these fields in dedicated sections, but you can have an overview of them [here](#) if you are curious. For information, you can also create custom fields to add additional metadata. So far, I never have the utility to doing that but if it's relevant according to your needs, you will find some guidelines [over there](#).

8 - Licensing

Choose your licence for your package allows you to declare how you want your code to be used and an important step if you use or include code written by someone else, to respect his work and license associated. You don't need, thankfully, to be an expert and you just need to follow several guidelines. If you have any doubt or if you are in a specific case, like if selling your package, don't hesitate to consult juridic service of your company/institute or for example a lawyer.

8.1 Global overview

There are 3 main kind of licenses when you talk about software development: the open source, proprietary and source-available licenses. If we go beyond software development, you can find other families of licences in relation to the kind of creation. We can mention [Creative Commons](#) (CC) family for creative works (art, photography, music, ...), [Open Data Commons](#) (ODC) licences for data and databases licences or the [GNU Free Documentation License](#) for Documentation and Educational Content Licenses for manuals, textbooks, tutorials, and academic materials. Here we are just focusing our reflection on software development family (with a little shortcut for licences dedicated to data) because we only deal in our example with an R package.

8.2 Software development licenses

Focusing on this kind of creation, three major families were available.

The first one is open source licences. These licenses allow users to access, modify and redistribute the source code freely. They promote transparency, collaboration, and innovation.

The second one is proprietary licences. These kind restrict access to the source code and limit how the software can be used, modified or distributed. The aims under that could be protection of commercial interests and maintain control over the software.

The last one is source-available licenses. These one make the source code visible but do not grant full open source freedoms like we could have with open source licences. This allow transparency while preserving business models or ethical boundaries.

For our use here, we mainly talk about open source licences because we will use the guideline of the [FAIR principles](#) to rule our work. We will see below how to specify a proprietary licence if necessary but I let you go deeper into these licences, and the source-available licenses, if you think your work needs one of them.

Part IV

Third part - Adding code

Part V

Fourth part - Tests and checking

Part VI

Fifth part - Documentation

Part VII

Sixth part - Current management

References

1. Wickham H, Bryan J (2023) R packages: Organize, test, document, and share your code (2nd edition). O'Reilly Media
2. Wickham H, Hester J, Chang W, Bryan J (2022) [Devtools: Tools to make developing r packages easier](#)
3. Wickham H, Bryan J, Barrett M, Teucher A (2024) [Usethis: Automate package and project setup](#)
4. Wickham H, Danenberg P, Csárdi G, Eugster M (2024) [roxygen2: In-line documentation for r](#)
5. Wickham H, Hesselberth J, Salmon M, Roy O, Brüggemann S (2025) [Pkgdown: Make static HTML documentation for a package](#)
6. Wickham H (2011) [Testthat: Get started with testing](#). The R Journal 3:5–10
7. Hester J, Angly F, Hyde R, et al (2025) [Linttr: A 'linter' for r code](#)
8. Ushey K, Wickham H (2025) [Renv: Project environments](#)
9. Csárdi G, Hester J (2025) [Pak: Another approach to package installation](#)
10. Yu G (2020) [hexSticker: Create hexagon sticker in r](#)
11. Csárdi G, Müller K, Hester J (2023) [Desc: Manipulate DESCRIPTION files](#)

A Not define yet

Appendice a